TF-IDF Lyric-Based Song Recommender

This notebook builds the baseline lyric recommender using **TF-IDF vectorization** and **cosine similarity**. The goal of this baseline is to measure how well direct lexical overlap in song lyrics can recover meaningful similarity between songs.

TF-IDF is a strong baseline because it gives higher weight to words and phrases that are distinctive within the corpus while down-weighting very common terms. In this model, songs are represented in the original lyric vocabulary space, so recommendations are driven by basic textual similarity. That makes the model easy to interpret: when two songs are recommended together, the similarity often comes from overlapping words, repeated phrases, or closely related lyrical language.

This baseline is important because it provides a clear point of comparison for the LSA recommender developed next. If TF-IDF captures literal lyrical resemblance, then the LSA model can be evaluated in terms of whether it moves beyond exact vocabulary overlap and captures broader semantic structure.

Data: Personal Spotify liked songs (~1928 songs with lyrics fetched via Genius API)

1. Imports & Setup

In [1]:
```
import sys
sys.path.append('../src')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

from tfidf_recommender import build_tfidf_recommender, recommend_songs

pd.set_option('display.max_colwidth', 60)
```

2. Load Data & Build Recommender

In [2]:
```
LYRICS_PATH = '../data/processed/lyrics_clean.csv'

lyrics_df, tfidf_matrix, similarity_matrix = build_tfidf_recommender(LYRICS_

print(f'Songs loaded:       {len(lyrics_df)}')
print(f'TF-IDF matrix:      {tfidf_matrix.shape}  (songs x vocab terms)')
```

```
print(f'Similarity matrix:  {similarity_matrix.shape}')
lyrics_df.head()
```

```
Songs loaded:       1502
TF-IDF matrix:      (1502, 10000)  (songs x vocab terms)
Similarity matrix:  (1502, 1502)
```

Out[2]:

	track_id	track_name	artist	lyrics	processed_lyrics
0	0qNnE0beu4OfHk8Txa3vGt	Soul Flower - Remix	The Pharcyde	[Chorus]\nEe, ah, ooh, ah, ah, ee, ah, ooh, ah, ah\nEe, ...	ee ah ooh ah ah ee ah ooh ah ah ee ah ooh ah ah ee ah oo...
1	7AHX0F99w4x23uSZihklZ6	My God	Tapir!	[Verse 1]\nWaking up with intuition\nGoing back to sleep...	waking intuition going sleep rumination premonition youl...
2	1KbpVvPiLAt1Wsh5gr2YKM	Take a Trip	TV Girl	[Intro]\nTake, take, take a trip, take a trip back down\...	trip trip memory lane trip trip memory lane hand pocket ...
3	56s4IHGnJVGZYcdOVmC3eb	I Love Life, Thank You	Mac Miller	[Intro]\nAlright then, bitch\nHa, ya\nUh\nA million moth...	alright bitch ha ya uh million motherfuckin far as biotc...
4	5XufqHphZkJhIkc5wlEST7	Soft Rock Band	Good Morning	[Intro]\nWell I played time in exile\nAnd the weather it...	played time exile weather hot dream recollection parking...

3. Vocabulary Exploration

Exploration of what the TF-IDF model captured in its vocabulary.

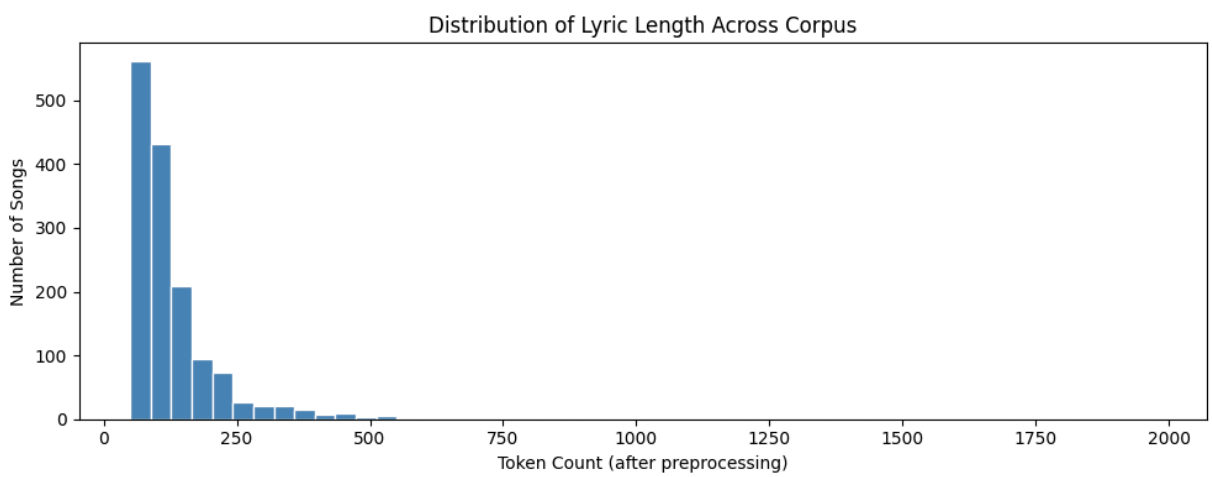
In [3]:

```
vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1, 2), min_df=
vectorizer.fit(lyrics_df['processed_lyrics'])

vocab = vectorizer.get_feature_names_out()
print(f'Vocabulary size: {len(vocab)} terms')
print(f'\nSample unigrams: {[t for t in vocab if " " not in t][:20]}')
print(f'\nSample bigrams:  {[t for t in vocab if " " in t][:20]}')
```

```
Vocabulary size: 10000 terms

Sample unigrams: ['10', '100', '11', '12', '13', '14', '15', '16', '18', '1
9', '1984', '1995', '20', '200', '2015', '2020', '2023', '2030', '21', '21s
t']

Sample bigrams:  ['21 savage', 'aap rocky', 'act like', 'actin like', 'actin
g like', 'admit ive', 'advice hey', 'afraid im', 'african american', 'ah a
h', 'ah baby', 'ah future', 'ah got', 'ah ha', 'ah know', 'ah let', 'ah oh',
'ah ooh', 'ah yeah', 'ahah ahah']
```

In [4]:

```
# Distribution of processed lyrics length
lyrics_df['token_count'] = lyrics_df['processed_lyrics'].apply(lambda x: ler

fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(lyrics_df['token_count'], bins=50, color='steelblue', edgecolor='whi
ax.set_xlabel('Token Count (after preprocessing)')
ax.set_ylabel('Number of Songs')
ax.set_title('Distribution of Lyric Length Across Corpus')
plt.tight_layout()
plt.show()

print(lyrics_df['token_count'].describe().round(1))
```




```
count    1502.0
mean      145.2
std       181.9
min        50.0
25%        76.0
50%       102.0
75%       147.0
max      1975.0
Name: token_count, dtype: float64
```

Plot Function for Word Similarity

In [5]:

```
def plot_top_terms(seed_title, seed_artist=None, n_terms=20, ax=None):
    matches = lyrics_df[lyrics_df["track_name"].str.lower() == seed_title.lo
    if seed_artist:
        matches = matches[matches["artist"].str.lower() == seed_artist.lower
```

```
    if matches.empty:
        raise ValueError(f"No match found for {seed_title} by {seed_artist}"

    seed_idx = matches.index[0]

    vec = TfidfVectorizer(max_features=10000, ngram_range=(1,2), min_df=2)
    matrix = vec.fit_transform(lyrics_df["processed_lyrics"])
    feature_names = vec.get_feature_names_out()

    scores = matrix[seed_idx].toarray().flatten()
    top_indices = scores.argsort()[::-1][:n_terms]

    top_terms = [feature_names[i] for i in top_indices]
    top_scores = [scores[i] for i in top_indices]

    if ax is None:
        fig, ax = plt.subplots(figsize=(10, 5))

    ax.barh(top_terms[::-1], top_scores[::-1], color='steelblue')
    ax.set_xlabel("TF-IDF Score")
    ax.set_title(f"Top {n_terms} Terms:\n'{seed_title}'")
```

4. Generate Recommendations

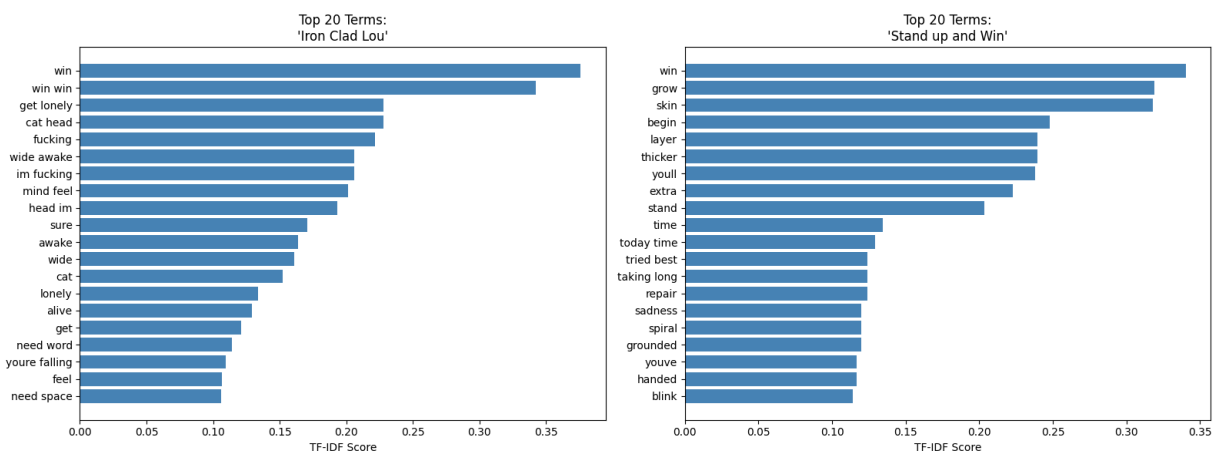
Testing recommender with a few seed songs, trying different songs to see how the system behaves across contents.

In [6]:
```
def show_recommendations(seed_title, seed_artist=None, k=10):
    """Wrapper that prints recommendations in a readable format."""
    recs = recommend_songs(
        lyrics_df, similarity_matrix,
        seed_title=seed_title,
        seed_artist=seed_artist,
        k=k
    )
    print(f"\nSeed: '{seed_title}'{' by ' + seed_artist if seed_artist else
    print('-' * 55)
    print(recs.to_string(index=True))
    return recs
```

In [7]:
```
# --- Test 1 from the 90s Grunge Era:
recs_1 = show_recommendations('Iron Clad Lou', seed_artist='Hum', k=10)
```

```
Seed: 'Iron Clad Lou' by Hum
--------------------------------------------------------
                   track_name                       artist  similarity
0            Stand up and Win                       Spymob    0.144103
1                Never Lonely                Space Mountain    0.134514
2                    Downtown                      Varsity    0.122517
3                        Rain  World's Experience Orchestra    0.119490
4                Faxing It In                      Smushie    0.111566
5                     Hippies                     flipturn    0.109453
6  Warmest Part of the Winter                      Voxtrot    0.105958
7                        2023                Blue Rain Boots    0.104926
8                       Alive                     Pearl Jam    0.103227
9               Down the Line                  Beach Fossils    0.089960
```

In [8]:
```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

plot_top_terms('Iron Clad Lou', seed_artist='Hum', ax=axes[0])
plot_top_terms('Stand up and Win', seed_artist='Spymob', ax=axes[1])

plt.tight_layout()
plt.show()
```

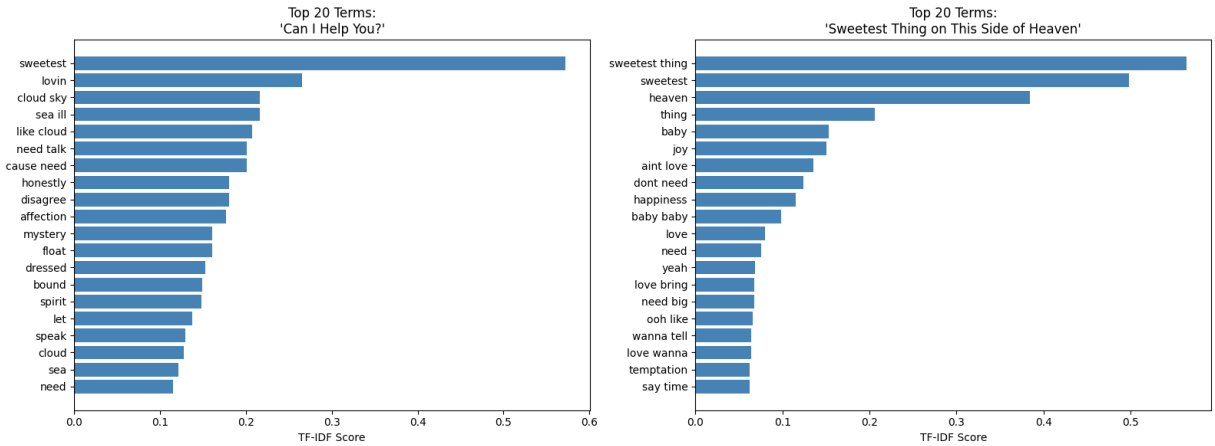


In [9]:
```
# --- Test 2: 70s Soul
recs_2 = show_recommendations('Can I Help You?', seed_artist="Amnesty", k=10
```

```
Seed: 'Can I Help You?' by Amnesty
--------------------------------------------------------
                              track_name                artist  similarity
0  Sweetest Thing on This Side of Heaven  Papa Bear & His Cubs    0.307253
1                               Chicago       Michael Jackson    0.171402
2                                Bailee             The Licks    0.160622
3                     Wizard Of Finance            Parliament    0.148595
4                          Cause = Time   Broken Social Scene    0.143805
5                          Mystery Girl             Housecall    0.123078
6                                 Ellie          Make Friends    0.118270
7                          Hard To Handle      The Black Crowes    0.097709
8                   Hey, Who Really Cares         Linda Perhacs    0.094847
9                          Emerald Eyes         Fleetwood Mac    0.091088
```

In [10]:
```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

plot_top_terms('Can I Help You?', seed_artist="Amnesty", ax=axes[0])
```

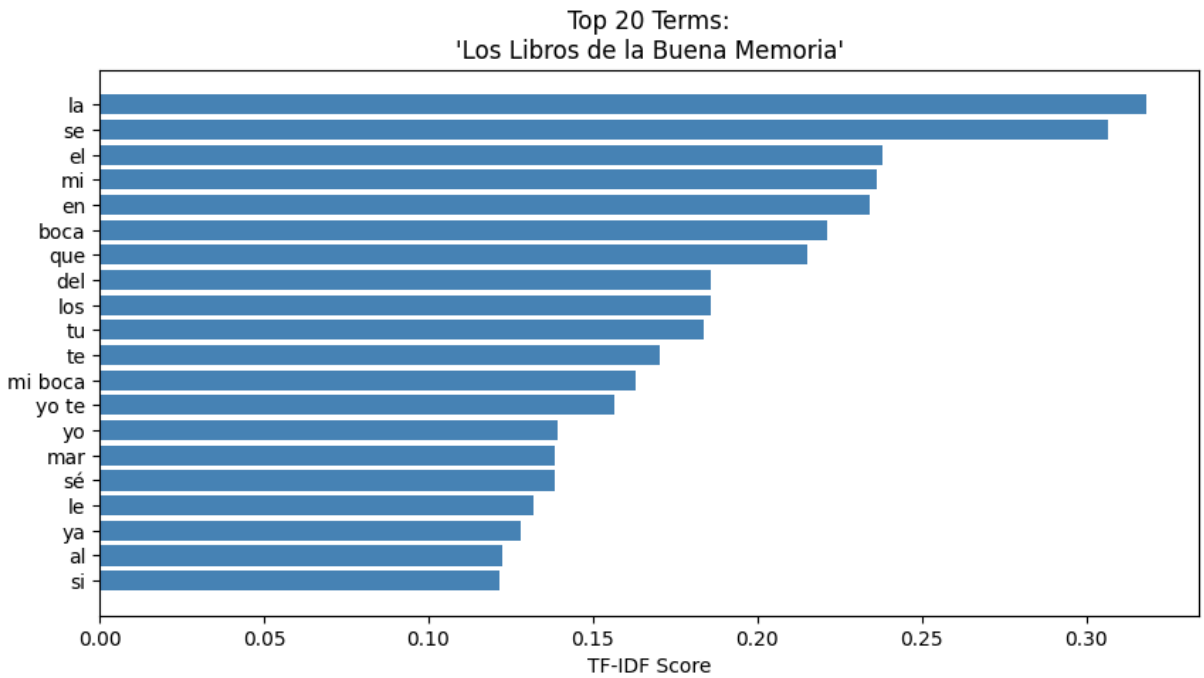
```
plot_top_terms('Sweetest Thing on This Side of Heaven', seed_artist="Papa Be

plt.tight_layout()
plt.show()
```



In [11]:
```
# --- Test 3: Latin Music
recs_3 = show_recommendations("Los Libros de la Buena Memoria", seed_artist=
```

```
Seed: 'Los Libros de la Buena Memoria' by Invisible
-------------------------------------------------------
                                                  track_name
artist  similarity
0                     SMUCKERS (feat. Lil Wayne & Kanye West)          Tyl
er, The Creator    0.497500
1                                Las Habladurias del Mundo
Pescado Rabioso    0.458267
2  After The Storm (feat. Tyler, The Creator & Bootsy Collins)
Kali Uchis    0.444739
3                                Como Mata el Viento Norte  La Maquina D
e Hacer Pájaros    0.426579
4                                                Oh Qué Será?
Willie Colón    0.421163
5                                  Promesas Sobre El Bidet
Charly García    0.364021
6                                    .—.. . — .. —  — —. — — —
. .—. .— ... . —..    0.358057
7                                      Elementales Leches
Invisible    0.353263
8                                                 La Balsa
Los Gatos    0.316008
9                                                 G-String
Jv    0.302299
```

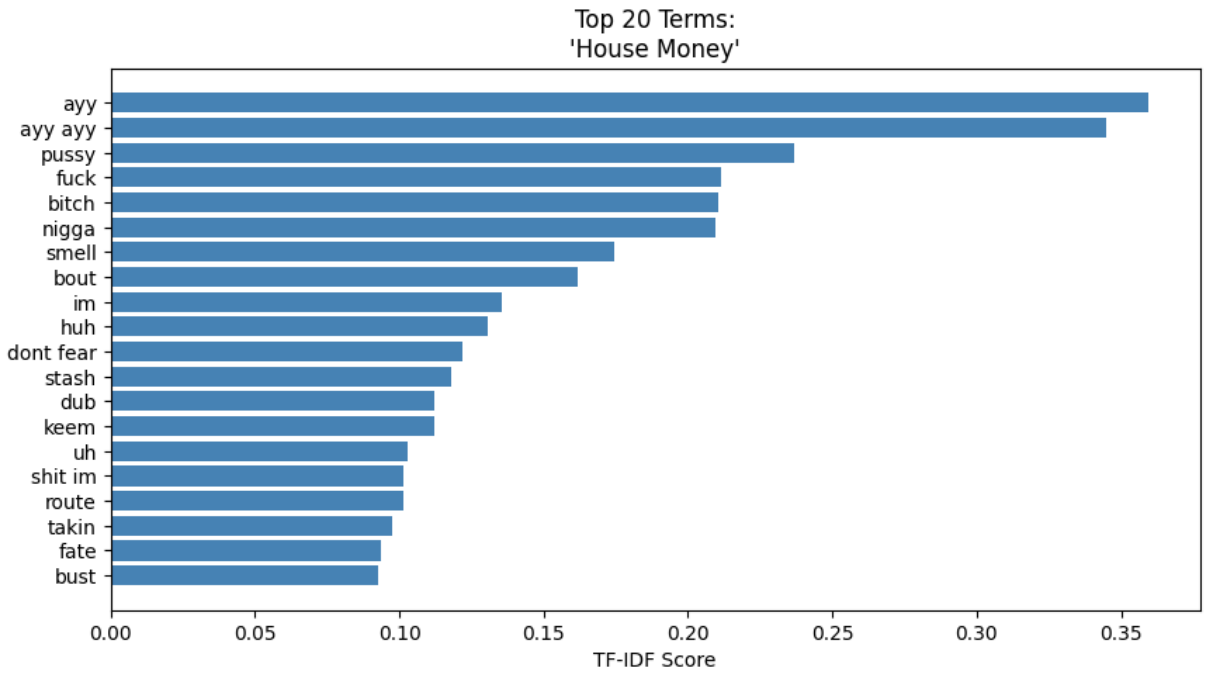
In [12]:
```
plot_top_terms("Los Libros de la Buena Memoria", seed_artist="Invisible")
```



In [13]:
```
# --- Test 4: Curent Hip-Hop
recs_4 = show_recommendations('House Money', seed_artist="Baby Keem", k=10)
```

```
Seed: 'House Money' by Baby Keem
--------------------------------------------------------
                            track_name                      artist  similari
ty
0  Jesus Is The One (I Got Depression)                    Zack Fox    0.4413
41
1                 Can You Rap Like Me?                Trippie Redd    0.3629
72
2                          Hit Bout It                  Lil Yachty    0.3292
07
3                          Blood On Me                      Sampha    0.3209
93
4                           REEL IT IN                       Aminé    0.2862
71
5                        trademark usa                   Baby Keem    0.2722
05
6                            JayJay-Z                   Zelooperz    0.2659
89
7                 Fresh Prince Of Utah  YoungBoy Never Broke Again    0.2624
20
8                               CORSO          Tyler, The Creator    0.2596
05
9                            WILSHIRE          Tyler, The Creator    0.2365
94
```

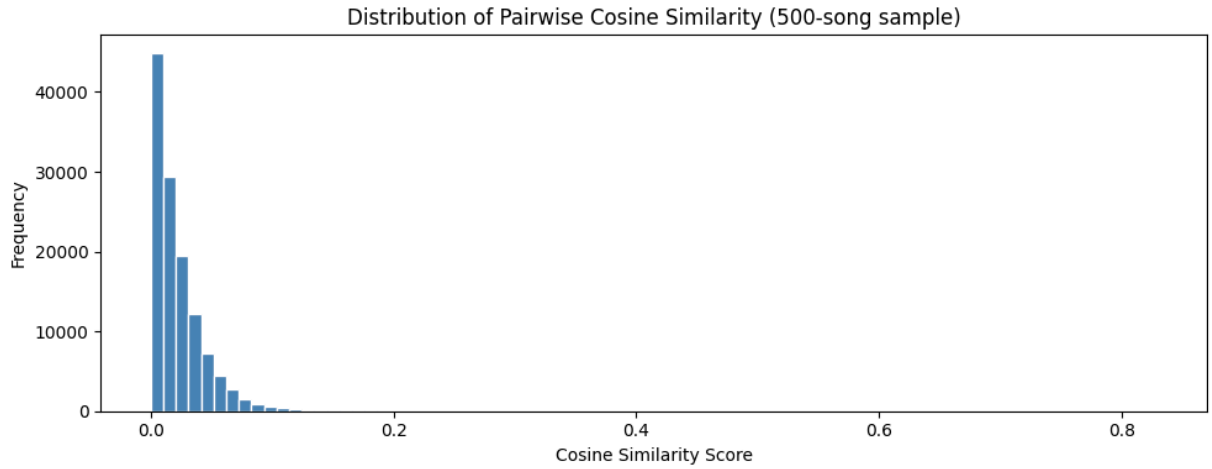
In [14]:
```
plot_top_terms('House Money', seed_artist="Baby Keem")
```



5. Similary Score Distribution

Looking at how similar songs generally are to each other in the corpus and whetehr the model is producing meanignufl seperation or mostly near-zero scores.

In [15]:
```
sample_size = 500
sample_indices = np.random.choice(len(lyrics_df), size=sample_size, replace=
sample_sim = similarity_matrix[np.ix_(sample_indices, sample_indices)]

# Extract upper triangle (exclude diagonal self-similarity)
upper = sample_sim[np.triu_indices(sample_size, k=1)]

fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(upper, bins=80, color='steelblue', edgecolor='white')
ax.set_xlabel('Cosine Similarity Score')
ax.set_ylabel('Frequency')
ax.set_title('Distribution of Pairwise Cosine Similarity (500-song sample)')
plt.tight_layout()
plt.show()

print(f'Mean similarity:   {upper.mean():.4f}')
print(f'Median similarity: {np.median(upper):.4f}')
print(f'Max similarity:    {upper.max():.4f}')
print(f'% scores > 0.1:    {(upper > 0.1).mean() * 100:.1f}%')
```




```
Mean similarity:   0.0226
Median similarity: 0.0160
Max similarity:    0.8290
% scores > 0.1:    1.3%
```

6. Most Similar Pairs in the Corpus

In [16]:

```
sim_copy = similarity_matrix.copy()
np.fill_diagonal(sim_copy, 0)  # zero out diagonal

top_flat = np.argsort(sim_copy.ravel())[::-1][:30]
top_pairs = np.unravel_index(top_flat, sim_copy.shape)

seen = set()
rows = []
for i, j in zip(*top_pairs):
    pair_key = tuple(sorted((i, j)))
    if pair_key in seen:
        continue
    seen.add(pair_key)
    rows.append({
        'Song A': f"{lyrics_df.loc[i, 'track_name']} — {lyrics_df.loc[i, 'ar
        'Song B': f"{lyrics_df.loc[j, 'track_name']} — {lyrics_df.loc[j, 'ar
        'Similarity': round(sim_copy[i, j], 4)
    })
    if len(rows) == 15:
        break

pd.DataFrame(rows)
```

Out[16]:

	Song A	Song B	Similarity
0	Come for Me — Twin Peaks	Lemon To My Lime — The Grogans	0.9075
1	Love the One You're With — The Isley Brothers	ramadan — ee	0.9027
2	Strawberry Letter 23 — The Brothers Johnson	Strawberry Letter 23 — Shuggie Otis	0.8991
3	Let Your Heart Hold Fast — Fort Atlantic	Cold Cold Man — Saint Motel	0.8923
4	For the Love of You, Pts. 1 & 2 — The Isley Brothers	Outer Space — Liz Cooper	0.8586
5	Bad Habit — Steve Lacy	I Have a Woman Inside My Soul — Yoko Ono	0.8524
6	Sleep Talking — Indigo De Souza	Besties — Black Country, New Road	0.8451
7	Lemon To My Lime — The Grogans	A Horse with No Name — America	0.8290
8	Lemon To My Lime — The Grogans	Cannibal Queen — Miniature Tigers	0.8269
9	Come for Me — Twin Peaks	Cannibal Queen — Miniature Tigers	0.8146
10	Sunday — The Moments	Lemon To My Lime — The Grogans	0.8132
11	Come for Me — Twin Peaks	A Horse with No Name — America	0.8119
12	São Paulo — Vincent Mango	Let Your Heart Hold Fast — Fort Atlantic	0.8084
13	São Paulo — Vincent Mango	Cold Cold Man — Saint Motel	0.7987
14	Sunday — The Moments	Come for Me — Twin Peaks	0.7958

Results

Overall, I think the simplicity of a TF-IDF model is highlighted here in these tests, but so is the task at hand. TF-IDF works well here as a baseline because it is highly interpretable: when two songs are similar, the reason is usually visible in repeated words, bigrams, chorus phrases, adlibs, or shared language. That strength is also its limitation. Across the corpus, similarity is extremely sparse, with a mean pairwise similarity of about 0.023 and a median of about 0.017, showing that most song pairs share very little direct lexical overlap. This means TF-IDF can do well when genres rely on repeated signature vocabulary, but it struggles when similarity depends more on tone, delivery, instrumentation, or broader semantic meaning than on exact wording.

Test 1: Grunge Music The TF-IDF model performed terribly on this test. It's highest similarity was 0.1506, primarily due to the recurring use of the word "win." In addition, only one of the top-10 most similar songs was actually considered grunge: Alive, by Pearl Jam. Yet, it was ranked as the 6th most similar. However, this tells us a lot about the genre. Grunge is the fusion of punk rock and heavy metal, charaterized by heavy guitar riffs, dark baselines, and the angst in which artists sings. The word frequency plot for this test's seed song show some of that lyrical angst, but the tone it is conveyed in proves to be is defining feature.

Test 2: Soul Music The TF-IDF model in its second test performed better, but still not well. It's highest ranked similarity was also a soul song, following similar themes around love, sweetness, and the need to express these feelings. However, similarites quickly dropped after rank one and only one other soul song was identified in the top-10. This was a more suprising result than the first test. Grunge music clearly relies on its hardcore rock vibes, and while soul music has its own music ecosystem too, I expected to see more simialrity present in the genre surrounding the genres lexicon.

Test 3: Latin Test This is was the TF-IDF model's most interesting results. First, we see a lot of high scoring Spanish langugae matches that are all Argentine progressive rock bands from the same era as Invisible. The model is picking up shared Spanish vocabulary. However, two of the top three most similar songs are hip-hop songs by the same artist: Tyler the Creator. This could be because of the numerous adlibs used in those songs that the model believes [adlibs] to be Spanish.

Test 4: Hip-hop Test This was were the TF-IDF model did its best. All of the top songs in the most similar rankings are hip-hop songs. This is likely due to the use of adlibs that are unique to rap and hip-hop, as well as the exessive use of explicatives.

The model performed like I expected. TF-IDF was strongest when songs shared distinctive repeated words, adlibs, or chorus phrases, and weakest when similarity depended more on musical style than on exact lyric overlap. The model did poorly on grunge, somewhat better on soul, interestingly on Latin songs through shared Spanish vocabulary, and best on hip-hop, where repeated adlibs and explicit lexical markers appear to be especially important. Across the full corpus, the similarity space remained extremely sparse, which confirms that exact word overlap between songs is relatively rare. That makes TF-IDF useful as a transparent baseline, but also shows why a stronger model is needed if the goal is to move beyond surface-level lexical resemblance.

A second important observation is that non-lexical vocables seem to matter a lot in this dataset. Repeated fragments like “la la la,” “doo doo doo,” or explicative phrases appear to inflate similarity between songs that may not actually be close in theme or genre. So

while TF-IDF is simple and interpretable, it can over reward catchy repeated lyric fragments and under represent deeper meaning.